AgentProf: Semantic Profiler for Long Horizon AI Agents

Anonymous submission

Abstract

AI agents produce growing populations of execution histories, and developers need population-level answers about where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Unlike stable code paths, natural-language intent lacks shared identity and native execution nesting provides no reusable cross-run hierarchy of semantic responsibility; existing tools trace runs, group metadata, and aggregate metrics but do not recursively construct such responsibility while retaining LLM/tool evidence and conserved measures in a standard profile. Agent observability needs profiling, not only debugging. AgentProf realizes this insight through a *semantic operation stack model*: backend-neutral recursive annotations identify shared responsibilities over an ordered source tree, operation stacks compose them with native evidence, and count, time, token, file, and network views replay fixed hierarchies into pprof-compatible profiles. We evaluate AgentProf on 405 human-staged CodeTraceBench trajectories, three complete public localization benchmarks, 287 OSWorld-Human sessions, three population case studies, and four public cost workloads. Direct Agent annotation raises ordinary B^3 F1 against human stages from 0.541 for raw action and 0.663 for recurrence to 0.764. On three complete localization workloads, AgentProf raises MAP over benchmark-native direct diagnostics by 0.031, 0.107, and 0.117. As a reading index on TraceElephant, the semantic hierarchy guides a strong trajectory reader to equal ranking quality while opening only 53.0% of the source evidence—versus 65.0% for an information-matched raw-action grouping at the same quality—and skeleton-guided drilldown remains available beyond the context window where whole-trace reading fails. Evaluated task-family and action backends reach 0.695 and 0.498 macro-F1, label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1 on OSWorld-Human, and, after marks are fixed, AgentProf constructs a 27,765-operation profile in 1.16 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, tool invocations) and a low-level system-effects layer (process, file, and network I/O), and production teams accumulate many such trajectories.

Developers lack population-level answers about agent quality, safety, and cost: which task categories consume the most token budget, where failures concentrate across workflows, and which behavioral patterns trigger unsafe system effects. Manual trajectory inspection is slow and costly (Lù et al. 2025), and LLM judging needs a separate evaluator pass per run (Lù et al. 2025; Zheng et al. 2023).

Agent trajectories lack two structures needed for cross-run profiling: responsibility lies in semantic categories such as task intent and tool operation rather than stable function names, and natural-language prompts provide no common identifier for shared intent. Native execution nesting may exist but provides no reusable cross-run hierarchy of semantic responsibility: one prompt sequence may represent multiple tasks, and one task may belong to a larger one.

Existing agent observability lacks this profiling abstraction: tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025); some derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026), and recent research canonicalizes actions into process profiles or cross-run graphs (Shu et al. 2026; Liu et al. 2026a; Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026; Gao et al. 2026; Wang et al. 2026b). None combines recursively annotated semantic responsibility, source-linked LLM/tool evidence, conserved additive measures, and standard profiler output over the same histories.

Agent observability needs profiling, not only debugging. Our *semantic operation stack model* provides this structure by projecting resource measures onto stable semantic tags in an attribution hierarchy: source nodes retain native session, prompt, LLM-call, and tool-call relations and additive measures; an interchangeable backend names the session scope, each user-prompt scope, and recursively refined intervals as operations; an *operation stack* emits the agent, those operations, and covered LLM/tool evidence. Raw session and prompt IDs stay labels, not visible frames that fragment aggregation (Figure 1).

We implement this model in AgentProf, an offline profiler compiling local agent trajectories into pprof-compatible profiles (Google 2024) (Figure 2). *Opera-*

tion annotation lets an Agent, language model, or non-LLM backend mark nested source intervals with shared semantic names, and *stack construction* deterministically emits `agent`→session-level operation→prompt-level operation→recursive operations→LLM call→tool call, retaining raw session and prompt identities as pprof labels for drilldown.

Across 405 released CodeTraceBench trajectories reconstructable from source (Li et al. 2026), direct Agent annotation reaches 0.764 ordinary B^3 F1 against human stages, vs. 0.663 for multi-resolution recurrence and 0.541 for raw action. Across three complete public workloads, AgentProf refines benchmark-native direct diagnostic ties and raises MAP by 0.031, 0.107, and 0.117, respectively (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a). On TraceElephant, the same fixed hierarchy serves as a reading index: a strong reader reaches the same ranking quality while opening less source evidence than an information-matched raw-action grouping (53.0% versus 65.0%). On 287 OSWorld-Human sessions (Abhyankar, Qi, and Zhang 2025), label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1. On the AgentBoard and ASE populations, task-family and action backends reach 0.695 and 0.498 macro-F1, respectively (Ma et al. 2024; Bouzenia and Pradel 2025). On four public workloads, after marks are fixed, AgentProf constructs a 27,765-operation profile in 1.16 s.

This paper makes three contributions:

1. **Semantic operation stack model.** Recursive semantic responsibility over source-native evidence supplies agent observability's missing profiling layer (Background and Motivation; Design).
2. **System: AgentProf.** A backend-neutral annotation workspace composes recursively marked semantic intervals with source-native hierarchy and produces pprof-compatible profiles (Implementation).
3. **Evaluation.** Automatic operation structure improves human-stage agreement on CodeTraceBench, and operation profiles supplement benchmark-native direct diagnostics on all three localization workloads while tying an information-matched raw-evidence refinement. A repeated-task resource case and a population-scale differential case expose high-cost unfinished work and success–failure differences. After marks are fixed, AgentProf profiles 27,765 operations in 1.16 seconds (Evaluation).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves intent-layer activity (prompts, LLM calls, and tool invocations for code execution, search, and file editing) with lower-layer process, file, and network effects (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). Trajectories repeat this cycle, and production teams accumulate many trajectories (Zheng et al. 2025).

System Profiling

Unlike single-execution debugging, profiling aggregates measurements to attribute resources to responsible entities: traditional profilers sample execution, attach each sample to its call stack, and fold identical stacks (Google 2024; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google 2024) call graphs visualize aggregate cost by width or node size, Pivot Tracing (Mace, Roelke, and Fonseca 2015) groups causally related measurements, and these tools start from stable code identifiers and runtime stacks, although pprof can also promote `tagroot`/`tagleaf` values to pseudo-frames (Google 2024). Agent profiles must instead derive stable semantic fields and connect them to downstream effects.

A Motivating Case: Two Flame Graphs

Figure 1 draws two standard pprof profiles with one renderer. The upper profile is a Go HTTP service under load, captured by stock `runtime/pprof`. The lower profile aggregates three independent agent executions of one Git-deployment task, run by two agent frameworks over two models.

Both panels are read by the same rules: a frame's width is its aggregate cost, a parent's cost contains its children's, and identical stacks recorded at different moments fold into one frame. The upper panel folds repeated executions of `main.handleIngest` across thousands of requests; the lower panel folds one `diagnose authentication` responsibility across three separate runs. What differs is where the frame names come from. A call stack supplies `net/http.(*conn).serve` for free at every sample. Nothing in an agent trajectory supplies `diagnose authentication`: the three runs express that intent through different prompts and different shell commands, and the name exists only because the profiler constructed it.

That constructed name is what makes the lower panel answerable. The three runs spend 4.56 million tokens, and nearly half of that mass falls under one shared authentication responsibility that both OpenHands executions return to after control and fallback attempts, although none of the three runs ever establishes the password-authenticated `git@localhost` path the task asked for. Read as raw trace records, the same work is 105 separate calls spread over six action kinds, 102 of them merely `run`, interleaved with 97 unrelated operations. The Evaluation section measures this case, its same-input organization controls, and whether the effect holds at population scale.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; OpenTelemetry 2024) record agent events as per-execution span trees for single-run debugging; AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating extended Berkeley Packet Filter (eBPF) system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Agent profiling faces two challenges. First, responsibility lies in semantic categories such as task intent and workflow phase, not code paths. Standards represent application-supplied span attributes (OpenTelemetry 2024; Arize AI 2024), but natural-language histories lack stable categories because different prompts or tools can express the same intent.

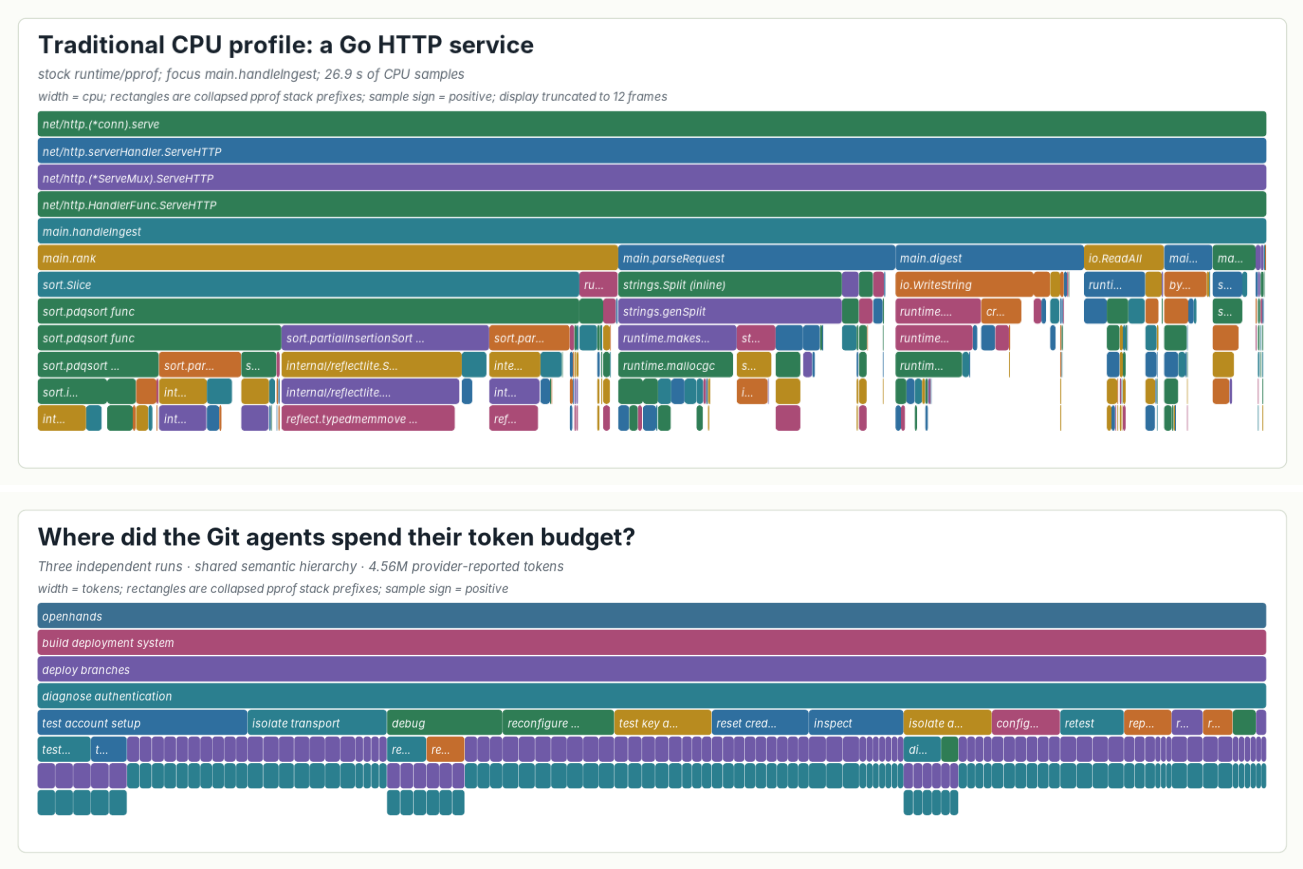

Figure 1: Two standard pprof profiles under one renderer. Top: CPU time in a Go HTTP service, recorded by stock `runtime/pprof` and truncated to twelve frames for display. Bottom: provider-reported tokens across three independent executions of one Git-deployment task, produced by AgentProf. In both panels width is aggregate cost, a parent's cost contains its children's, and identical stacks fold; only the frames' meaning differs—library and application functions above, constructed semantic responsibilities below.

Second, agent trajectories lack a runtime call hierarchy for semantic attribution. In CPU profiling, the stack attributes `malloc`'s cost simultaneously to `parse`, `process`, and `main` (Google 2024; Gregg 2011). A prompt sequence may contain one or several tasks within a larger workflow, yet no runtime mechanism marks its boundaries or attribution granularity. RQ1 evaluates how this missing hierarchy affects attribution, while RQ3 evaluates constructed groups and boundaries.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack supplies automatically. Agent profiling requires all three without a call stack:

D1: Conserved multi-resource attribution. Each atomic source record must retain additive resource measures, such as tokens, time, file effects, and network effects, so regrouping never changes their total mass.

D2: Shared semantic identity. Responsibilities that recur in different sessions need the same short semantic name so that their costs fold together rather than remaining session-local.

D3: Hierarchy without evidence loss. The profiler must derive variable-depth semantic responsibility from the native trace while retaining covered LLM/tool evidence as leaf frames and raw session/prompt identities as drill-down labels.

Design

The design has three explicit objects: an ordered source tree, nested semantic annotations, and a weighted pprof stack. A source adapter constructs the first without discarding content or native parent relations; any annotation backend constructs the second without rewriting source nodes; the deterministic profiler validates both, emits agent and semantic-operation frames followed by covered LLM/tool evidence, and folds equal paths into the third.

This separation addresses D1–D3 directly: additive measures stay on source nodes, shared semantic names create cross-session identity, and unique session, prompt, call, and event IDs remain pprof labels. Switching from operation

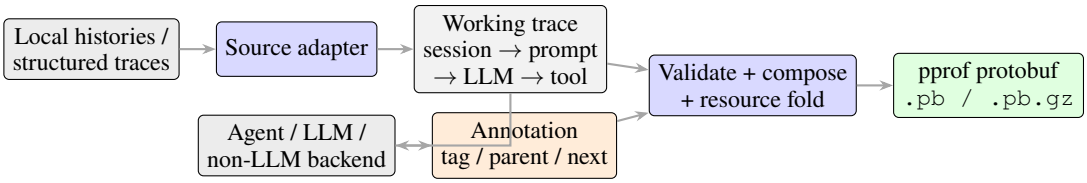

Figure 2: AGENTPROF pipeline. A source adapter preserves the native trace tree. An Agent, LLM, or non-LLM backend writes the same small annotation contract. The CLI validates and composes semantic paths with source evidence, folds one additive resource measure, and emits only a standard pprof profile.

count to tokens, time, file effects, or network effects replays the same annotation and changes only stack width.

Semantic Operation Stack Model

Let a source node be $n = (id, parent, kind, data, m)$ in an ordered forest. The *kind* is a session, prompt, LLM call, tool call, effect, or an adapter-defined atomic event. The *data* retains the source content needed to interpret the event, and m maps resource names to nonnegative additive measures. Source-parent links encode observed execution structure; they never encode an inferred semantic operation.

Recursive annotation covers every source node with exactly one active semantic leaf and its enclosing operation ancestors; let this path be $A(n)$, $g(n)$ the agent identity inherited from the source root, and $E(n)$ the retained evidence ancestry after removing raw session and prompt containers—normally the covered LLM call, tool call, and effect. The emitted operation stack is $\sigma(n) = \langle \textit{agent} : g(n) \rangle \parallel A(n) \parallel E(n)$ ($\parallel$ denotes concatenation), with raw session and prompt IDs as labels $\lambda(n)$ rather than visible frames, so equal prefixes aggregate across sessions without losing the evidence to locate a contributing call or tool event.

We formalize a resource profile as (φ, σ, w_r): predicate $\varphi(n)$ selects source nodes, $\sigma(n)$ supplies the composed stack, and $w_r(n) = m_n[r] \geq 0$ selects one additive resource. Folding merges equal stacks and sums w_r. The default operation-count view assigns one unit to each adapter-defined atomic operation; token, time, file-effect, and network-effect views select their corresponding measures. Because A and E remain fixed, changing r changes only widths, not boundaries or semantic names.

Recursive Operation Annotation

The source adapter preserves an ordered tree of sessions, prompts, LLM calls, and tool calls. Semantic operations are nested intervals over this tree rather than replacements for its nodes. An annotation at source node s contains a semantic name, the start node of its enclosing annotation, and the first source node after its interval. The exclusive end may be omitted when the interval reaches its parent's end. Every source root must begin a session-level operation, and every prompt must begin a prompt-level operation. Further nested intervals define the variable-depth suffix of the operation path. Equal path prefixes fold across sessions; covered LLM/tool nodes remain below the path, while raw session and prompt IDs remain labels for evidence drilldown.

Annotation proceeds coarse to fine. A backend first names the complete session and each user-prompt scope, then selects one interval whose current name hides a useful responsibility change, marks source-visible boundaries, names the resulting child intervals, and repeats only where another distinction helps; a later iteration may rename, merge, or remove an earlier split. The default Agent-assisted backend reads source content and existing paths to make these decisions. A plain language model, deterministic rules, change-point methods, and the label-free recurrence constructor use the same annotation contract, so the profiler does not depend on one model or inference service.

The evaluated Agent backend reads each trajectory's complete source-only packet once and directly emits sparse complete-path marks at the transition points it identifies, naming every enclosing responsibility. It chooses depth freely per branch, with no cap or threshold. One ordinary format retry is allowed, after which the deterministic root-prefix repair and canonicalization replay remain unchanged downstream. The complete backend interface is this input/output contract:

> *Input:* one trajectory's source-only packet—each turn's visible prompt, command, and output summary; no labels.
> *Action:* read once; emit a mark wherever the responsibility changes.
> *Output:* sparse complete-path marks, e.g.
> `turn 1: [deploy git server]`
> `turn 4: [deploy git server, diagnose SSH access]`
> `turn 9: [deploy git server, diagnose SSH access, test PAM]`
> `turn 15: [deploy git server, verify web endpoint]`
> —one line only where the path changes; free depth; names are one to three action-first words.

The complete CodeTraceBench direct-backend run uses independent Codex Agent workers under one fixed source-only annotation instruction. Each worker reads its trajectory once, with official stages and outcomes unavailable until all 405 trajectories are materialized; recurrence assignments and scores are likewise hidden. Validation is followed by the unchanged deterministic root-prefix representation repair (the appendix (*Operation-Identity Canonicalization*)) and source-only canonicalization replay before folding.

Implementation

Input reconstruction. AGENTPROF is an offline Rust CLI implementing this model (Figure 2). Source-specific adapters

reconstruct an ordered tree from local Codex and Claude Code histories, AgentSight (Zheng et al. 2025) recordings, or public trajectory datasets, preserving replay-stable node IDs, source content or readable references, native parent links, input order, and additive measures. Structured operation JSONL is materialized as adapter-defined atomic source nodes without inventing missing prompt or call relations.

Annotation workspace. The backend reads `trace.jsonl` and writes only `annotation.json`: the former retains source content, source-parent links, additive measures, and the current derived path; the latter maps source start IDs to `tag`, semantic `parent`, and exclusive `next`. AgentProf validates that references exist and intervals are nested, noncrossing, and collectively cover every source node, then atomically updates derived paths and the folded aggregate. Nonblocking hierarchy warnings flag a recursive operation with only one explicit semantic child, a large flat fan-out with little recursive refinement, or an optional semantic leaf spanning at least eight tool calls—exposing redundant, prematurely flat, or coarse annotations without forcing artificial depth or blocking generation.

Non-LLM recurrence backend. The inducer scores adjacent action transitions by normalized pointwise mutual information (Bouma 2009) and calibrates a label-free cutoff with occurrence-weighted two-means (MacQueen 1967), treating detailed recurrence as continuity that can remove but never add a coarse boundary; an optional reference-calibrated mode fits one scalar cutoff on disjoint reference groups (the appendix (*Recurrence Backend Details*)).

Profile export. AgentProf emits one pprof protobuf profile for existing tools such as `go tool pprof`; it does not add a visualization frontend or a second export format.

Privacy. AgentProf runs entirely offline on local histories; profiles carry only short semantic names, bounded text previews, and numeric measures as labels; no trajectory content leaves the machine unless the user shares the profile, and packet previews are truncated as disclosed in the appendix.

Evaluation

Research questions. The evaluation addresses four research questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Each RQ subsection pairs its protocol with the strongest answer its evidence supports. Together they form one chain: semantic responsibility can be recovered accurately (RQ3), the recovered hierarchy aggregates across runs while conserving every additive measure (RQ1), the aggregate corresponds to real problems and reduces the evidence a reader must open (RQ2), and construction and replay costs are measured and practical (RQ4).

We evaluate three data classes: **real inputs (RQ1, cases)**—41 long-horizon coding-agent sessions, 440 mixed-outcome web-agent trajectories, and 42 development sessions from the authors' workstation; **annotated trajectories (RQ3, RQ4)**—15 real-agent or human execution families and 405 CodeTraceBench trajectories (20,866 operations) (Lù, Kasner, and Reddy 2024; Deng et al. 2023; Xu et al. 2025; Yao et al. 2022; Li et al. 2023; Qin et al. 2024; Yao et al. 2025; Yang et al. 2024; Li et al. 2024; Lu et al. 2025; Liu et al. 2026b; Lù et al. 2025; Chen et al. 2026b; Wang et al. 2025; Abhyankar, Qi, and Zhang 2025); and **problem-localization benchmarks (RQ2)** over 27,346 operations (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a), with the RQ4 union at 27,765 operations (Lù et al. 2025; Chen et al. 2026b; Abhyankar, Qi, and Zhang 2025; Wang et al. 2025). Per-workload mean operations per trajectory are 8.5 (AgentProcessBench), 13.9 (OSWorld-Human), 24.0 (HINTBench), 27.1 (TraceElephant), and 51.5 (CodeTraceBench), so benchmark trajectories are short-to-medium horizon while the 42-session workstation population—whose longest sessions span tens of hours—supplies the long-horizon regime.

RQ1: Does Semantic Profiling Improve Resource Attribution?

RQ1 asks whether semantic profiling improves resource attribution. We test one necessary consequence: whether a fixed semantic hierarchy reveals materially different bottlenecks when the additive resource measure changes. The source population contains 41 real long-horizon agent trajectories, 3,146 source-native turns, and 5,750 operations, including three independent executions of one Git-deployment task whose 735 source nodes receive 96 recursive annotations; the resulting hierarchy reaches semantic depth five without a fixed limit, and the CLI reports no unary or flat-fan-out warning and 27 advisory coarse-leaf warnings. This establishes the multi-resource attribution capability on a repeated real task.

Case Study 1: Long-Horizon Agent Diagnosis. This is the case previewed in the Background and Motivation section. A stock-pprof focus query retains all three executions of the repeated Git-deployment task rather than selecting one trace, and Figure 3 replays that one fixed hierarchy under both additive measures.

The focused task contains 489 operations and 4,558,192 provider-reported tokens from OpenHands/Claude, OpenHands/DeepSeek, and Terminus2/DeepSeek: by count, Terminus2 contributes 56.24% and OpenHands 43.76%, while by tokens OpenHands contributes 86.62%. The repeated `diagnose authentication` operation directly contains 97 operations and 1,936,828 tokens, and its complete recursive subtree contains 105 operations and 2,103,587 tokens (21.47% and 46.15% of the focused task). The same unchanged hierarchy maps all 489 evidence rows under a defined elapsed-time width and exactly conserves 3,982 seconds; the subtree holds 1,492 seconds, or 37.47% of elapsed time. Its attributed share therefore rises from 21.47% by count through 37.47% by time to 46.15% by tokens: changing only the measure moves the share by more than a factor of two. Selecting the measure therefore changes which responsibility dominates: the count view attributes the task mainly to Terminus2, while the token view attributes nearly half of it to the authentication responsibility described in the

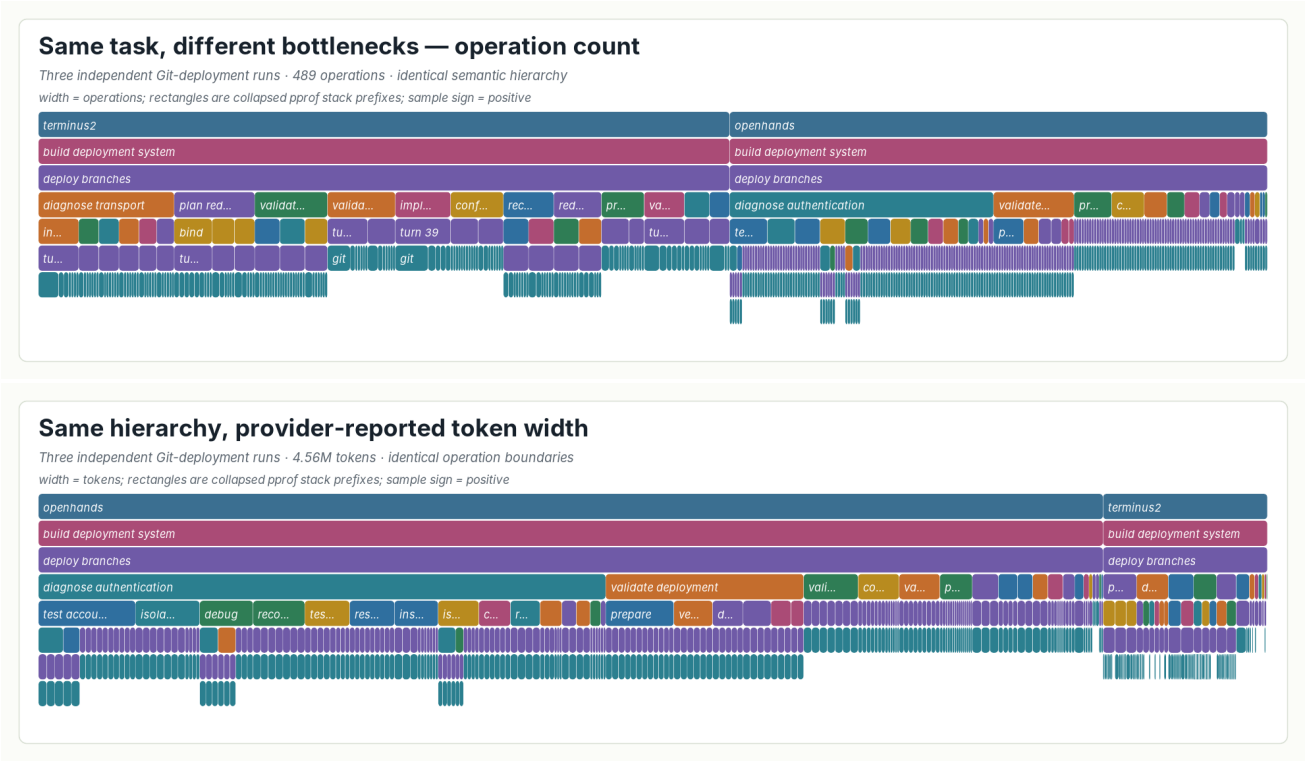

Figure 3: All three real `git-multibranch` executions under one fixed operation hierarchy, replayed under two additive measures: width is operation count (top) or provider-reported tokens (bottom). The hierarchy, its boundaries, and its names are identical in both panels; only the measure changes. Figure 1 focuses the shared `diagnose authentication` operation of the token view. AgentProf emits only standard pprof; a paper-only renderer reads the same `.pb.gz` through `go tool pprof` without introducing a product frontend.

Background and Motivation section, which source drilldown resolves to specific unfinished work. As a post-hoc organization control, we replay the same 489 source operations and both weights over common `project, agent` prefixes and `call, tool` leaves with three middle organizations—native (`session, prompt`), coarse (`action kind, raw action`), or semantic—so all six profiles conserve exact mass and open in stock pprof. The SSH members scatter across 105 native calls and span six coarse action kinds (largest branch 39.42% of their tokens), with 102 of 105 merely `run` alongside 97 unrelated operations. Only the semantic organization yields one focusable cross-run responsibility that an operation-count view understates; fixed from the case above, this control measures organization. This repeated real task supports RQ1's multi-resource attribution hypothesis: one shared responsibility path reunites the SSH work across executions, while the selected additive measure changes its attributed importance. Separately, real AgentSight eBPF recordings replay the same way: the R114 population folds all 1,520 process/file effects under task responsibilities through the recorded wrapper tool with exact conservation, demonstrating the system-effect layer end to end.

The same population-scale behavior holds on the frozen 440-session AgentRewardBench hierarchy (7,229 operations and 51,904,621 provider-reported tokens, both conserved exactly): replaying it ranks the same operations once by operation count and once by tokens. Over the 77 of 125 tasks with at least three distinct operations, mean per-task Kendall's tau-b (Kendall 1938) is 0.886 (10,000-draw task-cluster bootstrap 95% interval [0.857, 0.915]) and Spearman's rho (Spearman 1904) is 0.935 [0.917, 0.953]; the pooled population ranking agrees at tau-b 0.929. One fixed hierarchy therefore replays across measures with exact conservation and stable dominant responsibilities for most web-scale tasks, though 10 of the 77 rankable tasks fall below tau-b 0.7 and the long-horizon Git case above shifts attributed importance by more than a factor of two. Multi-measure replay is thus stable in the common case and decision-changing in a material minority—exactly the regimes where selecting the measure changes the engineering decision.

RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 asks whether target-blind profiles place independently annotated problems in groups a developer can inspect early. We run complete AgentProcessBench, HINTBench (the complete released test snapshot, 536 of the paper-reported 629 trajectories), and TraceElephant workloads (Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a); all zero-positive

Workload	Direct+ AgentProf	Direct+Raw +Evidence	Direct only	AgentProf only
AgentProcessBench	**.894**	.893	.863	.791
HINTBench	.517	**.518**	.411	.432
TraceElephant	**.326**	.324	.209	.259

Table 1: Standard per-query AP averaged as MAP over complete RQ2 populations. "Direct" is the benchmark-native trajectory judge/localizer. Both direct-first methods preserve every strict diagnostic ordering and retain identical source evidence. Higher is better.

trajectories are consumed for population coverage but excluded from MAP because AP is undefined without a relevant item.

Within each workload, operations, target-blind paths, benchmark predictions, and scoring are fixed before loading test targets (the appendix (*RQ2 Scoring Details*)).

The main test asks whether the profile adds information beyond the *direct diagnostic*—the per-operation output of a benchmark-native process judge (AgentProcessBench risk units) or trajectory localizer (HINTBench, TraceElephant), which for TraceElephant reads the full trace and reference answer before naming the responsible agent and decisive step. The candidate (Direct+AgentProf) ranks operations lexicographically by (*direct diagnostic*, *Agent+Evidence group score*), so it can only refine exact diagnostic-score ties. The other conditions are Direct-only, AgentProf-only (profile without the direct diagnostic), and Direct+Raw+Evidence (raw-action prefix; identical source-kind, tool/call, and outcome suffix and aggregation).

Each query is one trajectory containing an annotated target. We report non-interpolated per-query AP and arithmetic-mean MAP across 614, 400, and 220 target-bearing queries, respectively (Robertson 2008). All 522 zero-positive trajectories are consumed for population coverage but excluded from MAP because AP is undefined without a relevant item.

Direct+AgentProf raises MAP over Direct-only by .031 [.024,.039], .107 [.093,.120], and .117 [.088,.148], using 10,000 paired resamples of trajectory clusters within benchmark-defined strata. Thus the profile adds clear ranking information after a fixed trajectory reader on all three complete workloads (Table 1).

The information-matched Direct+Raw+Evidence refinement reaches .893, .518, and .324. Candidate-minus-baseline intervals are [-.0003,.0029], [-.0116,.0103], and [-.0247,.0280], so this experiment does not establish that the semantic-operation prefix ranks targets better when both views retain the same source evidence. The matched result attributes the ranking gain to group/evidence refinement in the complete profile, not specifically to its semantic prefix. The tie follows from the mechanism: per-operation anomaly signal resides in the retained source evidence, which both views share by construction. The semantic prefix's distinct, separately measured roles are cross-run attribution (RQ1) and directing a reader's attention, which the following study measures.

Profile-guided reading on TraceElephant. On the complete 220 target-bearing TraceElephant queries, a fixed external Grok-family CLI reader receives target-blind packets—task text, operation IDs, and source-visible content—with unranked operations appended in original order deterministically and one single-turn call per stage; full-trace reading reaches MAP .502 versus .209 for the benchmark's Direct-only diagnostic and .326 for Direct+AgentProf at a mean 12,615 input tokens per query. A two-stage profile-guided variant first shows only the semantic operation skeleton, with no source content, and the reader selects at most five groups; stage two then opens only the selected groups' evidence. It reaches MAP .455 while opening 53.0% of the source content, and its stage-one selections never fell back to a default; the five-group budget saturated on 99.5% of queries, and every selection miss was an absence from the reader's ordered choices rather than a budget cutoff—re-parsing the uncapped stage-one responses shows the reader proposed more than five groups on zero of 220 queries, and every missed target group was entirely absent from its ordered selection. Under an information-matched raw-action skeleton, ranking quality is statistically unchanged (MAP .465; paired delta $+.010$ $[-.021, +.042]$). But the reader opens significantly more content: 65.0% versus 53.0%, paired delta $+.120$ $[+.103, +.137]$, and 2.80 $[1.96, 3.60]$ more evidence operations. Semantic naming's measured contribution in this regime is attention concentration at equal quality. A per-query full read is bounded by the model context window: populations such as the 4,558,192-token repeated Git task cannot be read whole, whereas skeleton-guided drilldown remains available at any trace length. The reader is query-specific—it re-reads each trajectory per question—whereas the hierarchy is constructed once and replayed across queries and measures.

Case Study 2: Differential Profiling at Scale

We aggregate AgentRewardBench's (Lù et al. 2025) complete mixed-outcome population, not one favorable trace pair: 440 real trajectories over 125 tasks yield 338 bad–good pairs (24/102/144/68 from AssistantBench, VisualWebArena, WebArena, WorkArena), with the 202 successful and 238 unsuccessful sessions reusing across pairs (pair-occurrence weighted).

Before opening outcomes or expert annotations, three automatic Agent workers produce 2,131 sparse recursive annotations over all 7,229 source operations. The resulting hierarchy reaches semantic depth four before its LLM-call and tool-call evidence leaves. We then fold all 338 bad members and all 338 good members into one signed operation-count pprof, preserving the same source occurrences on both the recursive and fixed-chain projections.

Formally, the signed profile is the difference of two nonnegative profiles, $(\varphi, \sigma, w_r)_{\mathrm{bad}} - (\varphi, \sigma, w_r)_{\mathrm{good}}$; each side satisfies the model's nonnegative additive-measure definition, and pprof renders the subtraction as positive and negative sample values.

The aggregate contains 7,366 bad-side and 3,780 good-side operation occurrences. Recovery accounts for 44.6% of bad-side occurrences but only 12.0% of good-side occurrences (completion: 1.8% and 5.1%), and the recursive

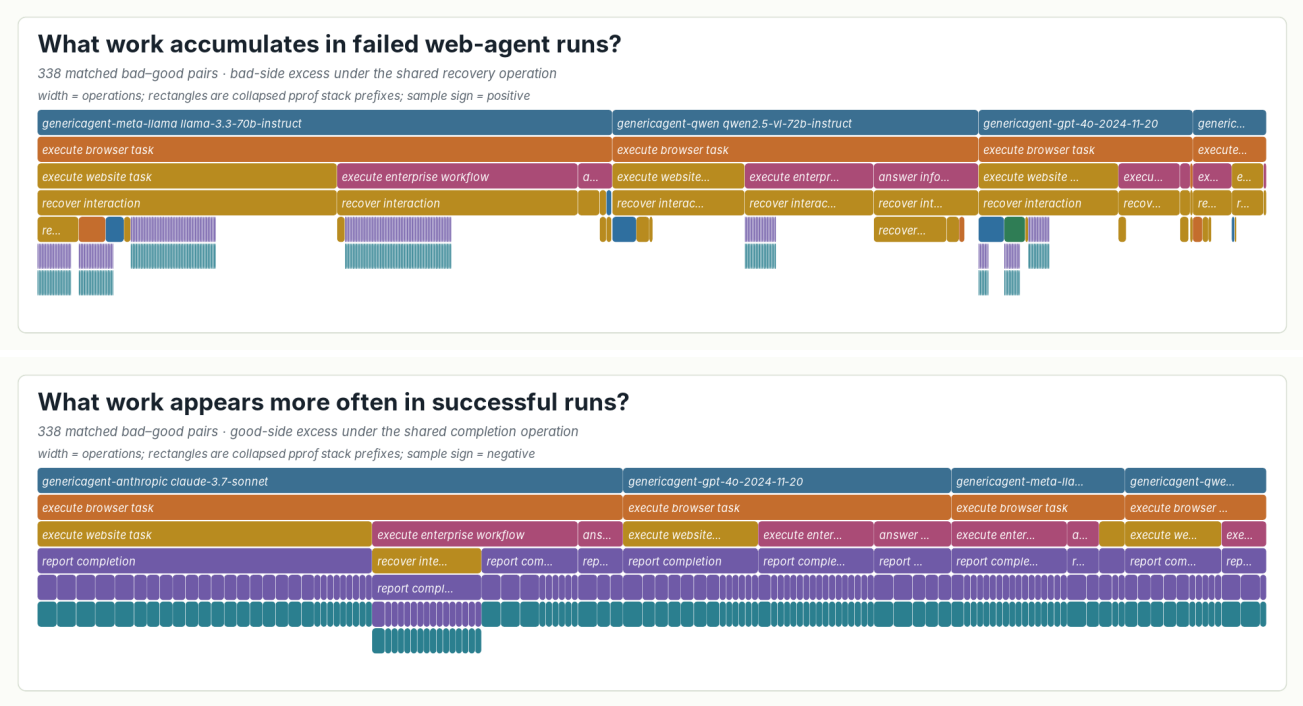

Figure 4: Focused paper renderings of the signed pprof profile over all 338 bad–good pairs. The top panel selects positive samples under `recover interaction`: 3,286 bad-side versus 455 good-side occurrences. The bottom selects negative samples under `report completion`: 135 bad-side versus 191 good-side occurrences. These are aggregate diagnostic differences, not causal effects.

recovery focus separates verification challenges, repeated searches, mistaken navigation, product or record retries, and repository inspection beneath the same responsibility, with source labels identifying the contributing session and call.

We test whether this visible recovery structure corresponds to an independent problem rather than merely reflecting more steps. For each trajectory, recovery exposure is the fraction of source operations whose path contains the shared recovery responsibility. Among 435 trajectories with consensus expert looping labels, this score obtains AP .634 at prevalence .398; a 10,000-draw task-cluster bootstrap places AP minus prevalence in [.181,.293]. A registered fixed-chain repeated/error projection obtains AP .656, and the recursive-minus-fixed interval [-.107,.061] does not establish detector superiority. The two projections answer different questions: the fixed indicator only flags where repeated/error steps occur, whereas the recursive profile decomposes the same responsibility into verification, search, navigation, retry, and inspection with session/call drilldown—a source-drillable explanation of *where* recovery accumulates that independently corresponds to expert-identified looping.

This case answers RQ2 at collection scale: a target-blind recursive profile exposes failed-side recovery and successful-side completion under shared responsibilities, and its recovery signal corresponds to an independently annotated real problem.

Case Study 3: Profiling the Agents that Built This Profiler

We profile all 42 development sessions recorded on the authors' workstation for this project—18 Codex and 24 Claude Code sessions produced by the agents that built AgentProf and this paper, including ten long-horizon sessions (Figure 5).

The longest sessions span tens of hours and hundreds of prompts. In this native no-sudo local-history scenario, one `-workspace-out` invocation initializes the standard annotation workspace directly from the raw session logs.

Across the population, the adapter materializes 10,423 source nodes: 42 sessions, 1,252 prompts, 5,620 LLM calls, and 3,509 tool calls carrying 1,380,863,014 bounded provider token components.

The fixed automatic instruction, identical to the AgentRewardBench run, makes one pass and produces 1,737 semantic annotations at depths two through four, covers all 1,294 mandatory session/prompt scopes, and incurs zero backend failures across 42 batches. The operation-count and token profiles conserve exact mass—3,509 and 1,380,863,014, respectively—and both load in stock pprof. The same self-profile hierarchy also replays under FILE-READ, FILE-WRITE, and NETWORK-target widths with exact conservation of 737, 31, and 61 target references, respectively (Figure 6). In the FILE-WRITE view, source-linked LLM and tool frames remain beneath each semantic operation, and

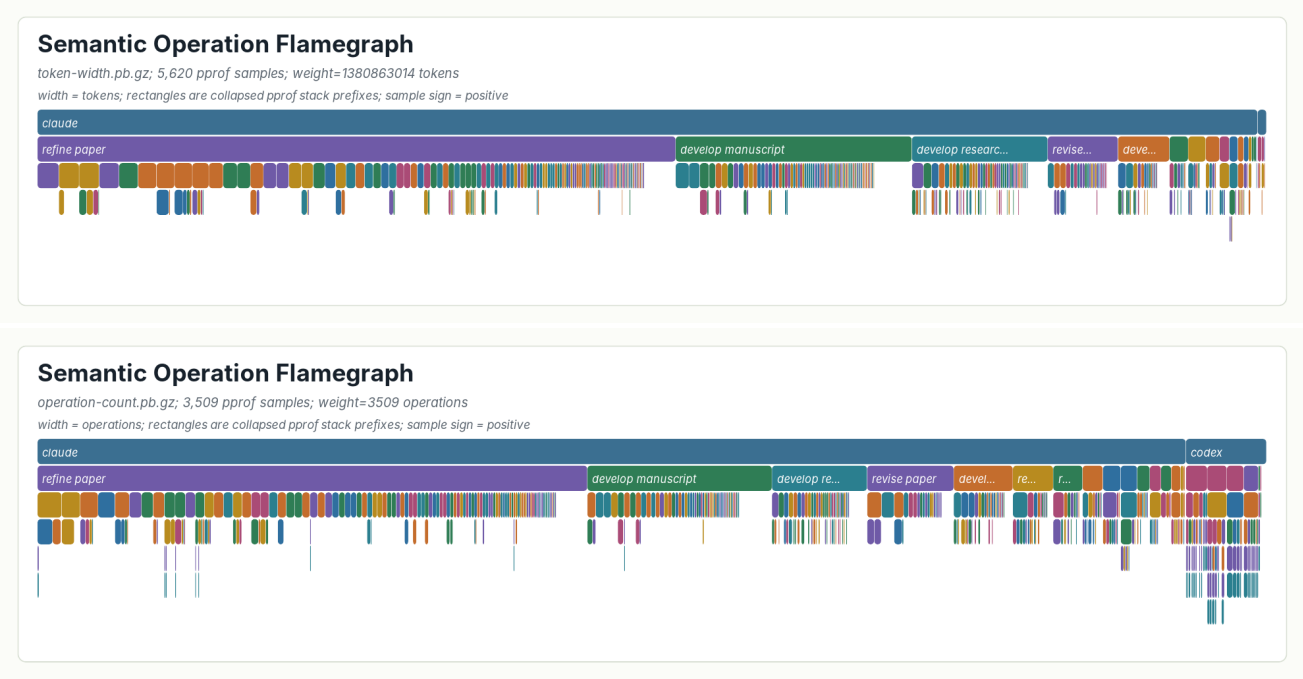

Figure 5: The complete 42-session population under one fixed semantic hierarchy. Width is provider-reported tokens (top) or operation count (bottom); second-level frames are the annotated responsibilities (`refine paper`, `develop manuscript`, `develop research`, ...), with deeper refinements and source labels available for drilldown in stock pprof.

each created file appears as an exact drilldown leaf beneath the operation that created it.

Development work is broad rather than concentrated: the largest token path, `refine paper > align evaluation`, holds only 1.735% of token mass. Token mass remains mostly at the mandatory prompt depth (70.4%), whereas operation mass resolves more deeply, with 43.9% at depths three and four. Most token mass staying at prompt depth reflects genuinely many-tasked development sessions under the fixed one-pass protocol; operation mass resolves deeper (43.9% at depths three and four) exactly where repeated engineering work concentrates. The three longest sessions have distinct dominant responsibilities—evaluation alignment, evidence inspection, and merge/conflict resolution—so the profile preserves long-session responsibility structure rather than averaging it away.

With three workers, the annotation critical path is 44.6 minutes and consumes 15,231,328 reported input tokens, of which 13,112,320 are cached, plus 311,097 output tokens. Validation completes in 0.211 seconds. This case establishes descriptive feasibility on real long-horizon sessions without outcome labels.

RQ3: How Accurate Are the Tags?

RQ3 evaluates automatic backend outputs against independent annotations under each protocol's stated input boundary. Literal task and action fields use accuracy and macro-F1 (the unweighted mean of per-class F1) (Lewis et al. 2004). Partition-valued task, phase, and group fields use V-measure (Rosenberg and Hirschberg 2007) or ordinary B^3, and adjacent group boundaries use exact precision, recall, and F1 (Ruokolainen et al. 2016). Literal labels, permutation-invariant partitions, and adjacent boundaries are distinct outputs. A backend output is evaluated at the level it predicts: literal names, permutation-invariant leaf partitions, or adjacent interval boundaries. For legacy field-valued datasets, a mechanical adapter converts each predicted group into the same interval-annotation contract before AgentProf folds the original additive weights.

We first evaluate the direct Agent-assisted backend on all 405 reconstructable CodeTraceBench trajectories (Li et al. 2026): 20,866 operations and 2,948 human-verified contiguous stages across four agent frameworks. Direct Agents read complete source-only packets once and emit sparse complete-path marks over 17,148 source-native turns, without access to official stages or outcomes before materialization. The direct annotation emits 4,496 marks with semantic depth one (3 marks), two (2,873), three (1,588), or four (32); no prompt requested a depth. The adopted marks' name replay maps 3,895 open-vocabulary operation IDs to 783 stable action-first canonical IDs, independently re-expanding all operations from the unchanged marks, preserving the temporal occurrence and adjacent-boundary vectors exactly, and leaving no adjacent display-path collision. Nonadjacent and cross-session equal names intentionally merge so recurring work folds in pprof.

Ordinary operation-level B^3 measures the induced leaf partition, while exact adjacent-boundary F1 measures transition placement. Both are standard metrics and assign every operation equal weight. The recurrence and source-native

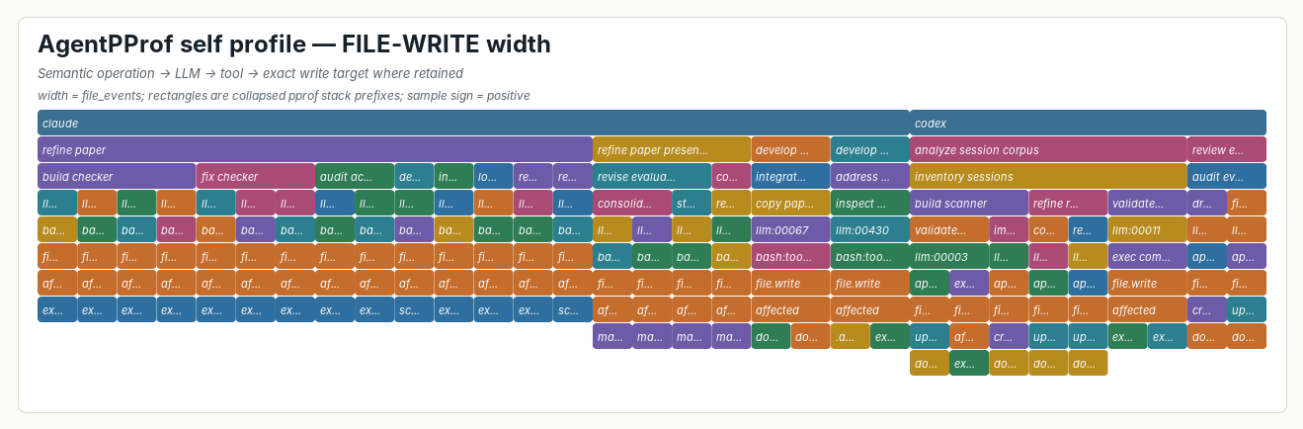

Figure 6: FILE-WRITE view of the same self-profile hierarchy. Width is target-reference count, with each created file preserved along the semantic operation → LLM → tool → exact write-target path.

Method	B^3 P	B^3 R	B^3 F1	Bound. F1
Direct Agent annotation	0.793	0.736	**0.764**	**0.480**
Multi-resolution recurrence	0.782	0.575	0.663	0.266
Native source tree	0.975	0.249	0.397	0.259
Source-native turn	0.983	0.221	0.361	0.246
Raw-action grouping	0.891	0.388	0.541	—

Table 2: Automatic operation-structure agreement on all 405 CodeTraceBench trajectories. B^3 scores the leaf partition; boundary F1 scores exact adjacent transitions.

controls receive the same operation sequence.

Direct Agent annotation reaches 0.764 B^3 F1, improving over recurrence by 0.101 (task-clustered bootstrap 95% interval [0.087, 0.116]). Its boundary F1 is 0.480 versus 0.266 for recurrence; the residual boundary error is over-segmentation rather than missed transitions (recall 0.626, precision 0.389), so the error therefore skews toward extra splits: predicted groups remain largely pure subsets of gold stages (B^3 precision 0.793), so the dominant failure subdivides work rather than merging unrelated responsibilities. Thus the direct Agent backend is the strongest tested automatic constructor on this complete population, and its marks conserve exactly 20,866 operations and 494,862,929 provider-reported tokens.

On all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations, the supervised predictor reaches 0.739 boundary F1 and 0.816 B^3 F1, label-free recurrence reaches 0.680 / 0.786, and the strongest control reaches 0.645 / 0.678 (the appendix (*OSWorld-Human Boundary Study Detail*)).

A target-blind TF-IDF/K-Means backend reaches V-measure (Rosenberg and Hirschberg 2007) 0.557 on nine Mind2Web sessions and 0.815 on 100 ScienceWorld sessions (Deng et al. 2023; Wang et al. 2022) (the appendix (*Partition Backends Detail*)).

For closed-label literal tags, a fixed evaluation-only Qwen3.6-27B backend (Qwen Team 2026) classifies all 1,012 AgentBoard goals (Ma et al. 2024) into nine task families at 0.695 macro-F1 and 0.733 accuracy versus 0.044 / 0.248 for the majority baseline, and a backend-level adapter classifies all 2,737 published ASE action labels from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Sajadi et al. 2026; Bouzenia and Pradel 2025) at 0.498 macro-F1 and 0.628 accuracy versus 0.061 / 0.323 for the majority baseline (the appendix (*Literal-Label Backend Detail*)).

Thus, across the named public populations, the direct Agent constructor reaches 0.764 B^3 and 0.480 boundary F1 on CodeTraceBench, the label-free recurrence backend reaches 0.786 B^3 and 0.680 boundary F1 on OSWorld-Human, and closed-label task-family and action backends reach 0.695 and 0.498 macro-F1, evaluating complementary structural and literal outputs rather than one bespoke score.

RQ4: What Is the Profiling Cost?

RQ4 separates fixed-input replay from the observable components of first profile construction. Across four complete public workloads and their union, we first measure the profiling core—parsing operation JSONL, constructing stacks, folding weights, and serializing the profile—by comparing the fixed semantic hierarchy `project, agent, task, phase, op, tool, status` with the raw hierarchy `project, agent, action, status` on identical inputs over three runs. Measurements use `agentpprof 0.2.37` on a 24-core Intel Core Ultra 9 285K with 125 GiB RAM and Linux 6.15.11.

Both time curves are monotonic across the five input sizes, and the semantic curve has a descriptive slope of 0.0418 ms per operation ($R^2 = 0.9997$), reaching 23,935 operations/s on the union (Table 3 in the appendix (*Cost Scaling Table*)). For the 27,765-operation union, semantic construction completes in 1.16 seconds, reaches 465.2 MiB peak RSS, and incurs 190 ms (19.6%) more time and 5.25 MiB (1.14%) more peak RSS than raw-action grouping. The table's AgentRewardBench row profiles a fixed 729-operation public release sample used by the cost harness; Case Study 2's 7,229-operation mixed-outcome popu-

lation is a different, larger workload. Constructing the 405 source-only CodeTrace packets used by the direct backend takes 501.64 s median; after annotation, its full deterministic downstream pipeline takes 11.516 s (the appendix (*Agent-Mark Reconstruction Cost Detail*)). For the complete 405-trajectory CodeTraceBench population, direct annotation consumes 12,050,384 input and 231,886 output tokens, with 2,215.858 s of active backend wall time.

A fully instrumented end-to-end annotation pass now exists. On the complete 440-session AgentRewardBench population, the fixed automatic backend completes all 12 outcome-blind batches in 3,521.6 s on a fixed two-worker schedule (58.7 minutes; summed worker time 6,661.7 s), consuming 12,039,417 actual input tokens (10,929,408 reported cached) and 312,433 output tokens—27,362 input and 710 output tokens per session. On the three-session Git population, one fresh complete pass takes 466.9 s and 832,544 actual input tokens. Deterministic materialization of the full 440-session population takes 0.26 s (operations) and 0.25 s (tokens): construction cost is dominated by the automatic backend, and materializing this population remains sub-second.

Scope details appear in the appendix (*Extended Scope and Limitations*).

Related Work

Agent observability. Platforms aggregate spans, metadata, and evaluations (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025); Datadog Patterns and LangSmith Insights derive cross-trace hierarchies and roll up metrics, NeMo profiles instrumented workflows, and OpenTelemetry Profiles links profiles to traces (Datadog 2026; LangChain 2026; NVIDIA 2026; OpenTelemetry 2026). AgentProf complements them by recursively assigning shared semantic responsibility over a preserved source-call tree and folding conserved resource measures into standard pprof.

Cross-run agent analysis. TraceProbe normalizes coding traces into canonical actions and process diagnostics, Graphectory builds cyclic action/navigation graphs and aggregates phase patterns (Shu et al. 2026; Liu et al. 2026a), Act·onomy supplies a fixed three-level behavior taxonomy, CHIEF builds task/subtask causal graphs for oracle-guided failure attribution (Gao et al. 2026; Wang et al. 2026b), Hodoscope compares behavior distributions to human review, and TraceGraph builds shared decision landscapes that guide recovery (Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026). AgentProf complements them with shared variable-depth responsibility annotations that cover native call evidence, fold across runs, and replay the same boundaries under different additive measures in a standard profiler. Concretely, four properties jointly separate AgentProf from each closest system: (i) variable-depth responsibility recovered from source content, where Act·onomy fixes a three-level taxonomy and TraceProbe canonicalizes a single action level; (ii) exact conservation of additive measures replayed across metrics, which the graph and distribution aggregations above do not provide; (iii) native LLM/tool evidence retained as drilldown leaves beneath every semantic frame, where causal-graph and distribution summaries discard call-level evidence; and (iv) standard pprof output consumed by existing tooling instead of a bespoke viewer. No compared system provides all four.

Profiling and diagnosis. pprof and flame graphs fold runtime stacks, whereas Pivot Tracing dynamically groups measurements across causally related events (Google 2024; Gregg 2011; Mace, Roelke, and Fonseca 2015). AgentRx diagnoses failures, CodeTracer reconstructs per-run states, and localization benchmarks score faulty steps, agents, or expert-supported perspectives (Barke et al. 2026; Li et al. 2026; Fan et al. 2026; Wang et al. 2026a; Chen et al. 2026a; In et al. 2026). AgentProf applies the profiler's population-level principle to agents, annotating recurring semantic responsibility over completed trajectories without code identity or treating runtime nesting as semantic truth.

Conclusion

Agent observability needs profiling, not only debugging. AgentProf recursively annotates semantic responsibility over source-native traces, folding covered evidence under shared cross-run operation stacks. Its profiles expose resource-dependent bottlenecks and success–failure differences, improve CodeTraceBench stage-partition agreement, supplement benchmark-native direct diagnostics on all three localization workloads, and recover OSWorld-Human groups at 0.680 boundary F1 and 0.786 B^3 F1. With marks fixed, AgentProf builds a 27,765-operation profile in 1.16 seconds, making population profiling practical alongside per-run debugging.

References

Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gao, J.; Sun, K.; Huang, J.-t.; Van Koevering, K.; Ji, S.; Huang, H.; Shi, W.; Lu, Z.; Xiao, Z.; Khashabi, D.; and Dredze, M. 2026. How to Interpret Agent Behavior. arXiv:2605.13625.

Google. 2024. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

In, Y.; Tanjim, M.; Subramanian, J.; Kim, S.; Bhattacharya, U.; Kim, W.; Park, S.; Sarkhel, S.; and Park, C. 2026. Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001.

Kendall, M. G. 1938. A New Measure of Rank Correlation. *Biometrika*, 30(1/2): 81–93.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.

Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.

Liu, S.; Chen, Y.; Krishna, R.; Sinha, S.; Ganhotra, J.; and Jabbarvand, R. 2026a. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1): 1961–1988.

Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.

Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML*, volume 235, 33007–33056.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Conference on Language Modeling (COLM)*.

Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362.

Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.

Nian, J.; Chen, K.; Zhang, G.; Cao, Y.; and Jiang, Y. 2026. TraceGraph: Shared Decision Landscapes for Diagnosing and Improving Agent Trajectories. arXiv:2605.31308.

NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.

OpenAI. 2025. OpenAI Codex CLI. GitHub repository.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

OpenTelemetry. 2026. OpenTelemetry Profiles. Official specification.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR conference on Research and Development in Information Retrieval*, 689–690.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420.

Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.

Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.

Shu, R.; Chong, C. Y.; Zhou, X.; Peng, Y.; Wu, Z.; Han, X.; Zhuang, Z.; Yuan, G.; and Wang, Y. 2026. What Resolve Rate Hides: Trajectory Structure Diagnostics for Coding Agents. arXiv:2607.06184.

Spearman, C. 1904. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*, 15(1): 72–101.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026a. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

Wang, Y.; Wu, W.; Wang, J.; and Wang, Q. 2026b. From Flat Logs to Causal Graphs: Hierarchical Failure Attribution for LLM-based Multi-Agent Systems. arXiv:2602.23701.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.

Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv:2604.11072.

Technical Appendix

Recurrence Backend Details

The Rust inducer counts adjacent action transitions in reference sessions and scores each transition by normalized pointwise mutual information (NPMI), $\mathrm{NPMI}(a,b) = \ln[p(a,b)/(p_L(a)p_R(b))]/-\ln p(a,b)$, with all three probabilities defined over the same transition sample space (Bouma 2009).

Occurrence-weighted one-dimensional k-means with $k = 2$ (MacQueen 1967) initializes at the minimum and maximum scores, assigns distance ties to the lower center, and uses the converged midpoint as a global cutoff. The inducer applies the same procedure to action-changing transitions for a second cutoff. Same-action pairs use the global cutoff, whereas different-action pairs use the smaller cutoff, which can remove a boundary produced by the global rule but cannot add one. When every reference and target operation also has a nonempty `action_detail`, the inducer fits the same recurrence model to the compound signature. Detailed continuity may remove a coarse boundary but can never add one. Missing, unseen, or weak detail falls back exactly to the coarse decision. Segment names remain coarse run-length-compressed action sequences. The inducer reads only session identity, input order, action, and optional action detail, not target annotations or resource weights.

An optional reference-calibrated recurrence mode uses the same NPMI score but selects one scalar cutoff that maximizes per-operation B^3 F1 on disjoint reference groups. The mode enumerates score intervals, resolves exact ties toward the smallest cutoff, and never reads target groups. Label-free k-means remains the default.

Operation-Identity Canonicalization

Before folding, one fixed source-only action–object map canonicalizes the Agent annotation's display identity. A boundary-safe refinement retains a canonical action and the minimum source words needed whenever two adjacent paths would otherwise become equal; unresolved collisions fail closed. Equal canonical names receive one stable operation ID across sessions. The complete replay reduces 3,895 open-vocabulary names to 783 two- or three-word identities with zero adjacent path collision, while preserving every temporal mark occurrence.

The adopted-mark representation repair attaches a one-turn root-only prefix to its first nested responsibility: when such a mark is immediately followed by its first responsibility, the first turn receives the complete root–responsibility path, eliminating artificial leaf groups without reading a stage label.

RQ2 Scoring Details

Within each workload, we fix operations, target-blind paths, benchmark judge/localizer predictions, and scoring rules before loading test targets. Group scores aggregate frozen benchmark-judge predictions per operation group (Wilson lower bound for HINTBench/TraceElephant; vote averaging for AgentProcessBench) (Wilson 1927).

Of HINTBench's reported 629 trajectories, the released test snapshot used here contains 536. A separate 80-trajectory validation snapshot selects one of 24 field orders before all 536 tests are scored.

AgentProcessBench averages judge votes within a group. HINTBench and TraceElephant assign each root-to-frame prefix the 95% Wilson lower bound (Wilson 1927) of its frozen positive-prediction fraction; an operation receives the maximum score over prefixes containing it.

Agent-Mark Reconstruction Cost Detail

The direct annotation backend is the OpenAI Codex CLI (version 0.145.0, model `gpt-5.6-sol`, default decoding, sandboxed non-interactive mode): one call per trajectory over the source-only packet, at most one format retry, and independent workers with no shared state. Packet construction bounds every text preview—prompt and response fields to at most 1,800 characters, tool commands to 600, and status previews to 200—while additive measures come from structured counters and are unaffected by preview truncation.

We reconstruct the source-only packet input three times on all 405 CodeTrace sessions (17,148 source-native turns and 20,866 operations). Joining the fixed target operations to released raw archives and constructing the packets takes 501.64 s median. The direct Agent run reuses those same packets and emits 4,496 adopted marks. Its complete deterministic downstream pipeline— annotation assembly, root-prefix repair, validation, name canonicalization, operation/token profile construction and stock-pprof readback, scoring, and paired analysis—takes 11.516 s. Both profiles load in stock pprof and conserve exact operation or 494,862,929-token mass.

Cost Scaling Table

Workload	Ops	Semantic (s)	Raw action (s)	Peak RSS (MiB)
AgentRewardBench	729	0.04	0.03	19.3
SATraj-OS	4,285	0.17	0.14	73.9
OSWorld-Human	6,010	0.24	0.21	109.1
AgentNet	16,741	0.70	0.58	279.3
Union	27,765	1.16	0.97	465.2

Table 3: RQ4 profile-construction cost. Times are three-run medians, and resident set size (RSS) is the largest peak observed for the semantic profile.

OSWorld-Human Boundary Study Detail

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations. This population contains 3,978 operations, 3,691 adjacent pairs, and 2,042 groups.

The supervised predictor reaches 0.739 boundary F1 and 0.816 B^3 F1 versus 0.645/0.678 for the strongest controls; label-free recurrence reaches 0.680/0.786 and reference-calibrated recurrence 0.734/0.801, the latter using group annotations the default label-free method avoids. Every method conserves the total projected weight.

Method	Prec.	Recall	Bound. F1	B^3 F1
Supervised predictor	0.700	0.782	**0.739**	**0.816**
Reference-calibrated recurrence	0.610	0.922	0.734	0.801
Label-free recurrence	0.592	0.799	0.680	0.786
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 4: RQ3 boundary and partition agreement on OSWorld-Human. Boundary F1 scores exact adjacent-pair decisions. B^3 scores predicted–human partitions.

For the supervised Bernoulli Naive Bayes predictor (McCallum and Nigam 1998), we fix the model family, nine input fields, adjacent-pair features, and training procedure. Each session-held-out cross-validation fold fits its parameters and threshold on training sessions and predicts every held-out session once. Predicted boundaries become an ordinary group field that AgentProf projects and folds. With the same five folds, label-free recurrence uses only the other sessions' visible action transitions. OSWorld-Human supplies no non-redundant action detail, so the multi-resolution constructor exactly falls back to its coarse arm. Optional reference-calibrated recurrence uses training operations and group annotations to fit one NPMI cutoff, while held-out folds contribute only visible actions until predictions are fixed. Controls place a boundary after every operation, visible action change, or visible phase change. We also report ordinary per-operation B^3 partition F1 (Bagga and Baldwin 1998), whose predicted–human group overlap captures merging and fragmentation.

Partition Backends Detail

For target-blind TF-IDF/K-Means, V-measure (Rosenberg and Hirschberg 2007) over operation assignments reaches 0.557 on nine Mind2Web sessions (49 operations) and 0.815 on 100 ScienceWorld sessions (2,504 operations) (Deng et al. 2023; Wang et al. 2022). Every operation receives a prediction in both datasets, whereas a constant tag yields V-measure 0. Here, V-measure tests partitions, not literal names.

Literal-Label Backend Detail

For task-family tags, we use a fixed evaluation-only Qwen3.6-27B llama.cpp backend distinct from the 3B CLI backend. Using only goals and declared family descriptions, it classifies all 1,012 AgentBoard goals into nine families (Qwen Team 2026; Ma et al. 2024). The backend reaches 0.695 macro-F1 and 0.733 accuracy, compared with 0.044 macro-F1 and 0.248 accuracy for the majority-class baseline, and produces identical assignments across three runs.

For action tags, a standalone backend-level adapter runs the fixed Qwen3.6-27B closed-label configuration using only the current thought/action and eight action definitions operationalized by TraceView (Qwen Team 2026; Sajadi et al. 2026). The adapter classifies all 2,737 published annotations from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Bouzenia and Pradel 2025). The model reaches 0.498 macro-F1 and 0.628 accuracy, compared with 0.061 macro-F1 and 0.323 accuracy for the majority-class baseline. Its 0.437 macro-F1 gain has a whole-trajectory 95% bootstrap interval of [0.380, 0.494], and both complete runs agree exactly. Thirty-nine AutoCodeRover inputs contain the target word `Locate`. Excluding them leaves 0.490 macro-F1 and 0.622 accuracy, so the positive comparison to majority is unchanged.

Extended Scope and Limitations

Results apply to the named populations and annotation protocols. RQ4 measures fixed-input replay, direct-backend wall time and token use, and the source-packet and deterministic downstream paths from normalized operations plus released raw archives. It excludes capture, raw-to-normalized conversion, and live-agent overhead; automatic annotation cost is instrumented on the complete CodeTraceBench, AgentRewardBench, and Git populations.

The direct annotation instruction was fixed before the CodeTraceBench run, the backend reads only source-visible content, and every stage label is loaded exclusively by the scorer after all 405 annotations materialize; no method component changed after scoring began. OSWorld-Human separately evaluates recurrence and supervised boundary backends. RQ2 targets are loaded only after profiles and predictions are fixed; RQ2 does not measure end-user diagnosis time. AgentRewardBench has no gold nested semantic hierarchy, so the differential case does not measure topology accuracy or causality.